

FINAL REPORT OF TRAINEESHIP PROGRAM 2024

Analysis of Chemical Components

A Content-Based Cosmetic Recommendation System

MedTourEasy

Data Analytics Traineeship

By: **Gracie Bhandari**

28 December 2024

Abstract

This project builds a content-based recommendation system for skincare products in which the "content" is each product's list of chemical ingredients. Its purpose is to help consumers, particularly people with sensitive skin, compare products objectively without needing a background in chemistry.

The analysis uses a Kaggle dataset of 1,472 Sephora skincare products across six categories and five skin types. During data cleaning, 171 products whose "ingredient list" was only a placeholder (for example "No Info" or "Visit the ... boutique") were removed, and ingredient names were normalized so that the same chemical is always counted once. For the worked example of moisturizers suitable for dry skin (171 products), the ingredient lists were tokenized into a binary document-term matrix of 171 products by 1,933 unique ingredients. Product similarity was measured with the Jaccard index, and t-SNE was used to project the products onto an interactive Bokeh map on which products with similar formulas appear close together.

The results show that ingredient overlap between most products is low (mean Jaccard similarity 0.063), so genuinely similar products stand out clearly. For example, AmorePacific's Color Control Cushion Compact (\$60) and LANEIGE's BB Cushion Hydra Radiance (\$38) share 23 ingredients (similarity 0.365, almost six times the average). The LANEIGE product is \$22 cheaper and better rated. A recommendation function ranks the most similar alternatives to any product, with an optional price limit, turning the analysis into a practical decision-support tool.

Table of Contents

Abstract	2
1. Introduction	4
1.1 About the Company	4
1.2 About the Project	4
1.3 Objectives and Deliverables	4
2. Methodology	5
2.1 Flow of the Project	5
2.2 Language and Platform Used	6
3. Implementation	7
3.1 Requirements and Problem Statement	7
3.2 Data Collection and Structure	7
3.3 Data Cleaning	7
3.4 Data Filtering	8
3.5 Tokenization and Document-Term Matrix	8
3.6 Similarity Measurement and t-SNE	8
3.7 Visualization and Recommendation	9
4. Code	10
4.1 Loading, Cleaning and Filtering	10
4.2 Tokenization	10
4.3 Document-Term Matrix and Similarity	10
4.4 t-SNE and Bokeh Map	11
4.5 Recommendation	11
5. Results and Observations	12
5.1 Ingredient Analysis	12
5.2 Similarity Between Products	12
5.3 Ingredient Similarity Map	13
5.4 Comparison of Two Products	13
5.5 Recommendations	13
6. Conclusion	15
Limitations	15
7. Future Scope	16
8. References	17

1. Introduction

1.1 About the Company

MedTourEasy is a global healthcare company that provides the informational resources people need to evaluate their global healthcare options, and offers analytical solutions to its partner healthcare providers. Its focus on applying data science to consumer-centric problems provided the setting for this project, which applies the same approach to the cosmetics domain.

1.2 About the Project

Buying a new cosmetic product is often difficult, especially for people with sensitive skin who are prone to adverse reactions. The information needed to make a good choice is printed on every product, but ingredient lists are long, technical and hard to compare without a chemistry background.

This project addresses that problem by treating ingredient lists as data. Each product is represented by the set of chemicals it contains, products are compared by how many ingredients they share, and the results are presented as an interactive map and as ranked recommendations. If a user likes one product, the system can point them to others with a similar chemical composition, often at a lower price.

1.3 Objectives and Deliverables

The objectives of the project are to:

1. **Build a recommendation system** that identifies products with similar chemical compositions.
2. **Simplify ingredient analysis** by turning long ingredient lists into an objective, comparable measure.
3. **Make exploration intuitive** through an interactive visualization that needs no chemistry background.
4. **Support personalization** by letting the analysis be run for any product category and skin type.

The deliverables are:

1. **Jupyter Notebook** containing the complete, reproducible analysis with outputs.
2. **Cleaned, filtered dataset** produced within the notebook, with a documented cleaning step.
3. **Interactive Bokeh map** of ingredient similarity with hover details, plus a static image of the map.
4. **Recommendation function** that returns the most similar products, with an optional price limit.
5. **Documentation**: this report and the project README.

2. Methodology

The project followed a structured pipeline that turns raw product listings into similarity scores, a visual map and recommendations.

2.1 Flow of the Project

1. **Problem definition:** identify the target users (people choosing skincare, especially those with sensitive skin) and the core problem of interpreting complex ingredient lists.
2. **Data collection:** a dataset of 1,472 Sephora skincare products was obtained from Kaggle.
3. **Data cleaning:** products without a usable ingredient list were removed, and unrated products were labelled as such.
4. **Filtering:** the data was narrowed to one product category and one skin type (moisturizers for dry skin in the worked example).
5. **Tokenization and document-term matrix:** ingredient lists were split into normalized ingredient names and encoded as a binary product-by-ingredient matrix.
6. **Similarity measurement:** the Jaccard index was computed between every pair of products.
7. **Dimensionality reduction:** t-SNE projected the products into two dimensions using the Jaccard distances.
8. **Visualization:** an interactive Bokeh scatter plot with hover details was created.
9. **Comparison and recommendation:** product pairs were compared quantitatively, and a function was written to recommend the most similar alternatives.

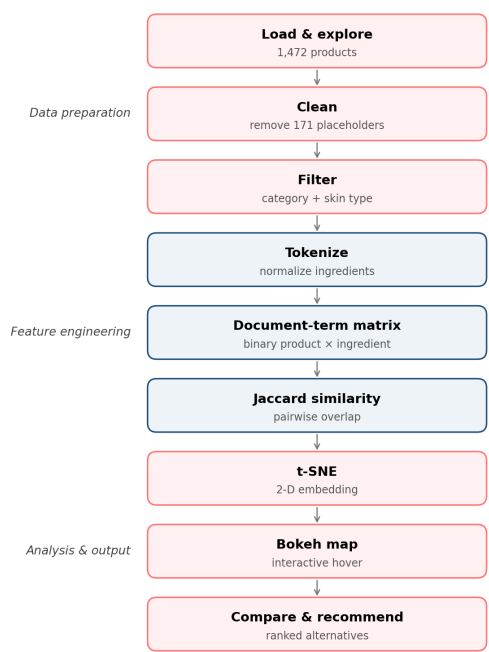


Figure 1: Project workflow

2.2 Language and Platform Used

Category	Tool	Purpose
Language	Python 3	Data preparation, analysis and visualization
Data handling	pandas, NumPy	Loading, cleaning, filtering and matrix operations
Similarity	SciPy	Pairwise Jaccard distances
Machine learning	scikit-learn	t-SNE dimensionality reduction
Visualization	Bokeh	Interactive similarity map with hover tool
Visualization	Matplotlib	Static figures for the README and this report
Environment	Jupyter Notebook	Reproducible, documented analysis
Data source	Kaggle	Cosmetics dataset (cosmetics.csv)

3. Implementation

3.1 Requirements and Problem Statement

Problem statement: consumers struggle to interpret and compare the chemical components listed on cosmetic products. The goal is a system that compares products by their ingredients and suggests similar alternatives.

Requirements:

- A dataset of cosmetic products that includes full ingredient lists and skin-type suitability.
- A reliable way to turn free-text ingredient lists into comparable features.
- A similarity measure suited to comparing sets of ingredients.
- An intuitive visualization and a ranked recommendation output.

3.2 Data Collection and Structure

The dataset (Dataset/cosmetics.csv) contains 1,472 products in a single table with the following columns. It has no missing values and no duplicate products.

Column	Type	Description
Label	Text	Category: Moisturizer, Cleanser, Face Mask, Treatment, Eye cream, Sun protect
Brand, Name	Text	Product brand and name
Price	Integer	Price in USD
Rank	Decimal	Average customer rating from 0 to 5 (0 means not yet rated)
Ingredients	Text	Comma-separated ingredient list as printed on the product
Combination, Dry, Normal, Oily, Sensitive	Binary	1 if suitable for that skin type, otherwise 0

Products per category: Moisturizer (298), Cleanser (281), Face Mask (266), Treatment (248), Eye cream (209), Sun protect (170). Products suitable for each skin type: Combination (966), Dry (904), Normal (960), Oily (894), Sensitive (756). A product can be suitable for several skin types.

3.3 Data Cleaning

Inspecting the ingredient column showed that 171 products (11.6%) do not contain an ingredient list at all:

Placeholder text	Products
"Visit the [brand] boutique"	120
"No Info"	23
"#NAME?" (spreadsheet error)	4
Footnote only (e.g. "*Plant origin.", "*Certified Organic")	24
Total removed	171

If these rows were kept, every placeholder product would appear to contain one shared "ingredient" (for example the text "no info") and would be wrongly placed next to unrelated products. After removing them, 1,301 products remain. In addition, 19 products have a rating of 0, which means "not yet rated" rather than a score of zero, so they are displayed as "Not rated".

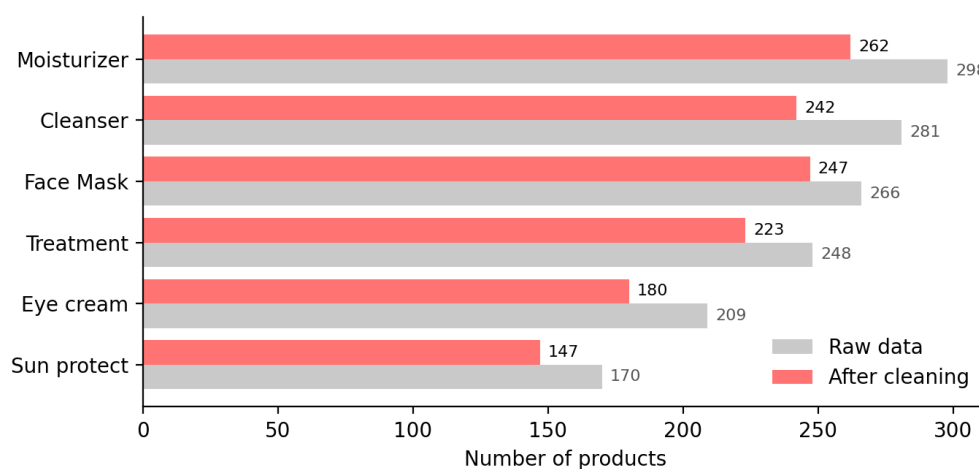


Figure 2: Products per category before and after cleaning

3.4 Data Filtering

The workflow is parameterized by two values, CATEGORY and SKIN_TYPE, so it can be run for any of the 30 combinations. The worked example selects moisturizers suitable for dry skin: 171 products after cleaning (190 before).

3.5 Tokenization and Document-Term Matrix

Each ingredient list is split at commas into individual ingredient names (tokens). Because the raw text is inconsistent, every token is normalized:

- converted to lower case, with extra whitespace collapsed and trailing periods removed;
- footnote markers (*) and a recurring retailer disclaimer removed;
- commas inside chemical names protected, so that "1,2-Hexanediol" stays one ingredient instead of becoming "1" and "2-hexanediol".

Without this step, "Citric Acid", "citric acid " and "citric acid." would be counted as three different ingredients. The tokens are then encoded in a binary document-term matrix of 171 products by 1,933 unique ingredients, in which a cell is 1 if the product contains that ingredient. On average a product contains 38.9 ingredients, so the matrix is very sparse (2.0% of cells are 1).

3.6 Similarity Measurement and t-SNE

Because each product is a set of ingredients, similarity is measured with the **Jaccard index**: the number of shared ingredients divided by the number of distinct ingredients in the two products combined. It ranges from 0 (nothing in common) to 1 (identical lists) and, unlike Euclidean distance on the raw matrix, is not biased by the length of an ingredient list.

t-distributed Stochastic Neighbor Embedding (t-SNE) was then fitted on the precomputed Jaccard distances to place every product in two dimensions so that products with similar

ingredients appear close together. A fixed random seed (42), perplexity of 30 and learning rate of 200 make the layout reproducible.

3.7 Visualization and Recommendation

The two-dimensional coordinates are plotted with Bokeh. Hovering over a point shows the product name, brand, price and rating, and the plot supports zooming and panning. A static version is also saved as an image for viewers that cannot run JavaScript, such as GitHub.

Finally, the function `recommend(name, top_n, max_price)` ranks all other products in the selection by Jaccard similarity to a chosen product and can restrict results to a budget.

4. Code

The complete code is in Notebook/cosmetic_recommendation_system.ipynb. The key steps are shown below.

4.1 Loading, Cleaning and Filtering

```
df = pd.read_csv(DATA_PATH)

PLACEHOLDER_PATTERN = r'^\s*(?:No Info|#NAME\?|Visit the .+ boutique|\.*)\s*$'
is_placeholder = df['Ingredients'].str.match(PLACEHOLDER_PATTERN)
df_clean = df[~is_placeholder].reset_index(drop=True)

CATEGORY = 'Moisturizer'
SKIN_TYPE = 'Dry'
products = df_clean[(df_clean['Label'] == CATEGORY)
                    & (df_clean['SKIN_TYPE'] == 1)].reset_index(drop=True)
```

4.2 Tokenization

```
DISCLAIMER_PATTERN = re.compile(r'please be aware that ingredient lists may change.*',
                                flags=re.IGNORECASE)
LOCANT_COMMA = re.compile(r'(?<![\w.])\d\s*,\s*(?=\d(?:,\d)*-)\s*')

def tokenize(ingredients):
    text = DISCLAIMER_PATTERN.sub('', ingredients.lower()).replace('*', '')
    text = LOCANT_COMMA.sub(r'\1\0', text) # protect commas in names like 1,2-hexanediol
    tokens = []
    for token in text.split(','):
        token = re.sub(r'\s+', ' ', token.replace('\0', ',')).strip()
        token = re.sub(r'(\d)-', ' ', token)
        if token and token not in tokens:
            tokens.append(token)
    return tokens

corpus = [tokenize(text) for text in products['Ingredients']]
ingredient_idx = {}
for tokens in corpus:
    for ingredient in tokens:
        ingredient_idx.setdefault(ingredient, len(ingredient_idx))
```

4.3 Document-Term Matrix and Similarity

```
M, N = len(products), len(ingredient_idx)

def oh_encoder(tokens):
    x = np.zeros(N, dtype=bool)
    for ingredient in tokens:
        if ingredient in ingredient_idx:
            x[ingredient_idx[ingredient]] = True
    return x

A = np.vstack([oh_encoder(tokens) for tokens in corpus]) # M x N matrix
jaccard_distance = squareform(pdist(A, metric='jaccard'))
similarity = 1 - jaccard_distance
```

4.4 t-SNE and Bokeh Map

```
tsne = TSNE(n_components=2, metric='precomputed', init='random',
            learning_rate=200, perplexity=30, random_state=42)
tsne_features = tsne.fit_transform(jaccard_distance)
products['X'], products['Y'] = tsne_features[:, 0], tsne_features[:, 1]

plot = figure(title='Ingredient similarity map', x_axis_label='t-SNE 1',
              y_axis_label='t-SNE 2', width=700, height=500)
plot.scatter(x='X', y='Y', source=source, size=10, color='#FF7373', alpha=0.8)
plot.add_tools(HoverTool(tooltips=[('Item', '@Name'), ('Brand', '@Brand'),
                                   ('Price', '$@Price'), ('Rating', '@Rating')]))

show(plot)
```

4.5 Recommendation

```
def recommend(name, top_n=5, max_price=None):
    idx = find_product(name)
    results = products.assign(Similarity=similarity[idx]).drop(index=idx)
    if max_price is not None:
        results = results[results['Price'] <= max_price]
    results = results.sort_values('Similarity', ascending=False).head(top_n)
    return results[['Brand', 'Name', 'Price', 'Rating', 'Similarity']]
```

5. Results and Observations

All results below are for moisturizers suitable for dry skin and are produced by the notebook.

5.1 Ingredient Analysis

The 171 products use 1,933 distinct ingredients, but a small set of base ingredients appears in most formulas: water (134 products), glycerin (128), phenoxyethanol (111), butylene glycol (94) and dimethicone (76). Most other ingredients appear in only a handful of products, which is what makes ingredient lists useful for telling products apart.

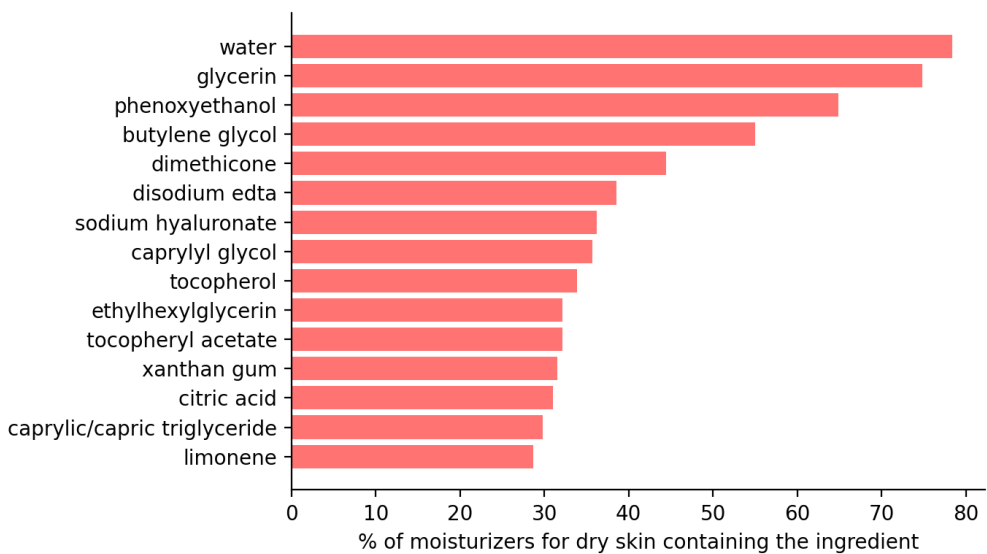


Figure 3: The 15 most common ingredients

5.2 Similarity Between Products

Most pairs of products share very little: the mean Jaccard similarity is 0.063 and the median is 0.062. Highly similar pairs are therefore rare and meaningful. The pairs with a similarity of exactly 1.0 are the same formula sold in different versions, for example La Mer Crème de la Mer and its mini, or Clinique Dramatically Different Moisturizing Lotion+ and its limited edition. This is a useful check that the method behaves as expected.

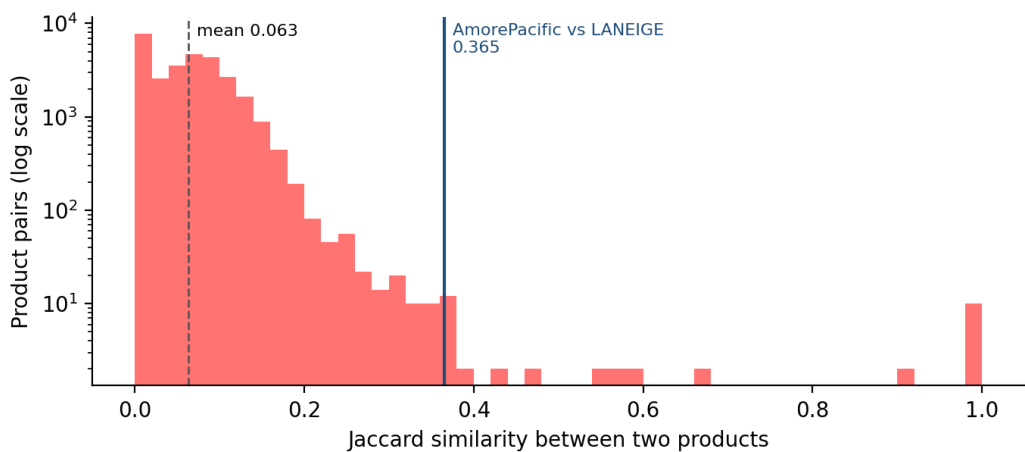


Figure 4: Distribution of pairwise Jaccard similarity

5.3 Ingredient Similarity Map

On the t-SNE map, each point is a product. The axes have no direct meaning; what matters is the distance between points, since products close together share a large part of their ingredient lists. Most products form one dense region. 8 products appear as isolated points because they share no ingredients with any other product. These are mainly single-ingredient oils (for example pure argan oil or 100% squalane) or listings that name only a few highlighted ingredients.

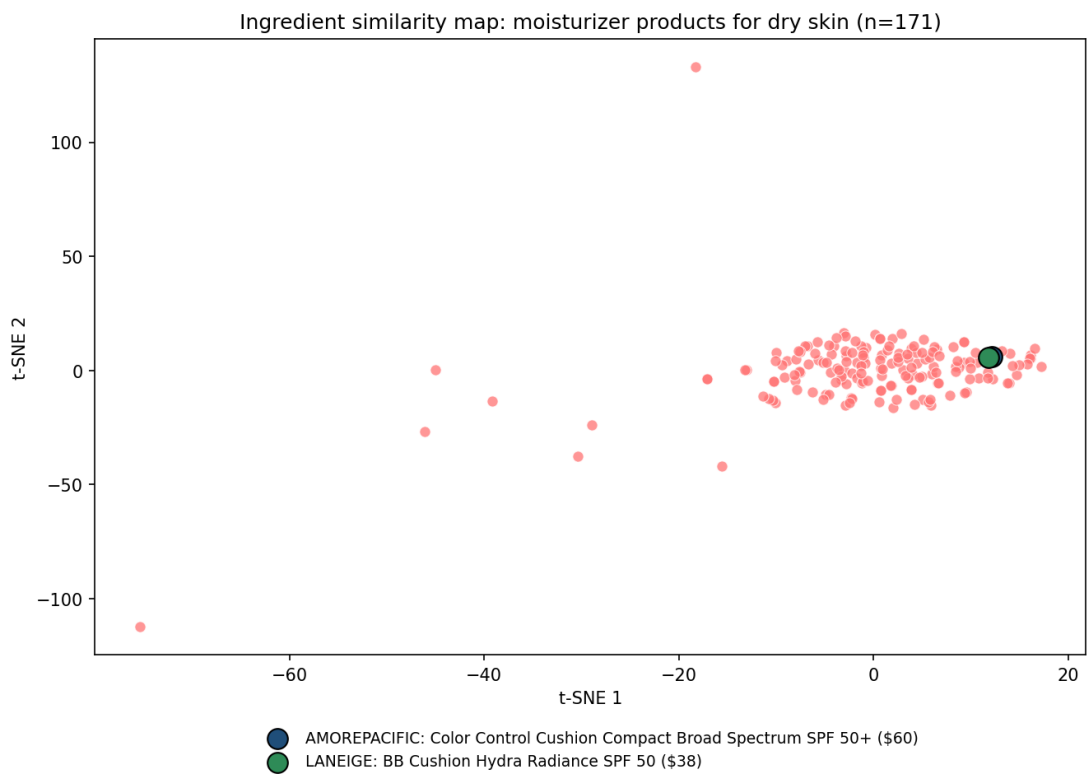


Figure 5: t-SNE ingredient similarity map with the two compared products highlighted

5.4 Comparison of Two Products

Brand	Product	Price	Rating	Ingredients
AMOREPACIFIC	Color Control Cushion Compact Broad Spectrum SPF 50+	\$60	4.0 / 5	41
LANEIGE	BB Cushion Hydra Radiance SPF 50	\$38	4.3 / 5	45

The two cushion compacts sit almost on top of each other on the map. They share 23 ingredients, giving a Jaccard similarity of 0.365, which is about 5.8 times the average similarity between products. Shared ingredients include the silicone base (cyclopentasiloxane, cyclohexasiloxane, phenyl trimethicone, PEG-10 dimethicone), the same acrylate film-formers, and the same preservatives. The LANEIGE product is **\$22 cheaper and rated higher** (4.3 vs 4.0), which shows how the system can help users find a better-value alternative.

5.5 Recommendations

Asked for the products most similar to the AmorePacific cushion, the recommender ranks the LANEIGE cushion first, with a similarity about twice that of the next candidate:

#	Brand	Product	Price	Rating	Similarity
1	LANEIGE	BB Cushion Hydra Radiance SPF 50	\$38	4.3 / 5	0.365
2	SUPERGOOP!	CC Cream Daily Correct Broad Spectrum SPF 35+ Sunscreen	\$34	4.4 / 5	0.189
3	LANCÔME	Visionnaire Advanced Multi-Correcting Cream	\$88	4.6 / 5	0.162
4	LANEIGE	Water Sleeping Mask	\$25	4.4 / 5	0.162
5	IT COSMETICS	Your Skin But Better™ CC+Illumination™ Cream with SPF 50+	\$38	3.9 / 5	0.156

With a budget filter, the recommender finds alternatives to the most expensive product in the selection, Fresh Crème Ancienne® (\$290), priced at or below the median price of \$52:

#	Brand	Product	Price	Rating	Similarity
1	CAUDALIE	Vine[activ] Overnight Detox Oil	\$50	4.5 / 5	0.143
2	KIEHL'S SINCE 1851	Daily Reviving Concentrate	\$47	4.7 / 5	0.138
3	REN CLEAN SKINCARE	Evercalm™ Overnight Recovery Balm	\$48	4.9 / 5	0.128
4	CAUDALIE	Vinosource Intense Moisture Rescue Cream	\$39	4.2 / 5	0.109
5	SMASHBOX	Photo Finish Primer Water	\$32	4.1 / 5	0.103

6. Conclusion

This project developed a content-based recommendation system that compares cosmetic products by their chemical components. Starting from 1,472 Sephora products, the data was cleaned of 171 entries without usable ingredient lists, and ingredient names were normalized so that each chemical is counted consistently. The ingredient lists were converted into a document-term matrix, compared with the Jaccard index, and visualized with t-SNE on an interactive Bokeh map.

The results show that the approach works. Products with genuinely similar formulas stand out clearly from the low average similarity, identical formulas sold in different sizes are correctly identified as identical, and the worked comparison found a cheaper, better-rated alternative to a premium product. The recommendation function turns the analysis into a practical tool, and because the category and skin type are parameters, the same workflow serves users with different skincare needs.

The project shows how standard data-science techniques (text processing, similarity measures, dimensionality reduction and interactive visualization) can make complex consumer information accessible, helping people make more informed purchasing decisions without needing to be chemists.

Limitations

- Ingredient similarity is not the same as skin compatibility. Concentrations, ingredient order, formulation and individual sensitivities are not modelled, so the results support a decision but are not dermatological advice.
- Prices, ratings and formulas reflect the Sephora listings at the time the dataset was collected.
- A few listings give only highlighted ingredients rather than the full formula, and a few contain commas inside parentheses, which add minor noise to tokenization.
- t-SNE preserves local neighbourhoods well, but the global positions of clusters and the axes are not meaningful.

7. Future Scope

1. **Weighted ingredients:** ingredients are listed in descending order of concentration, so weighting them by position (or with TF-IDF) would let major ingredients count more than trace ones.
2. **Ingredient safety analysis:** flag common irritants and allergens (for example fragrance, certain preservatives and essential oils) to support users with sensitive skin.
3. **User feedback and personalization:** combine ingredient similarity with users' own ratings and reactions to build a hybrid recommender that improves over time.
4. **Interactive application:** package the map and recommender as a web app (for example with Streamlit or a Bokeh server) with drop-down filters for category, skin type and budget.
5. **Up-to-date data:** refresh product data regularly from retailer listings so that prices, ratings and formulas remain current.
6. **Alternative embeddings:** compare t-SNE with UMAP, and evaluate clustering methods to identify families of similar formulas automatically.

8. References

1. van der Maaten, L., & Hinton, G. (2008). Visualizing Data using t-SNE. *Journal of Machine Learning Research*, 9, 2579-2605.
2. Jaccard, P. (1901). Étude comparative de la distribution florale dans une portion des Alpes et du Jura. *Bulletin de la Société Vaudoise des Sciences Naturelles*, 37, 547-579.
3. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
4. Virtanen, P., et al. (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17, 261-272.
5. McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*, 56-61.
6. Bokeh Development Team. Bokeh: Python library for interactive visualization. <https://docs.bokeh.org/>
7. scikit-learn documentation: sklearn.manifold.TSNE. <https://scikit-learn.org/stable/modules/generated/sklearn.manifold.TSNE.html>
8. Cosmetics dataset (cosmetics.csv): Sephora skincare products with ingredients and skin-type suitability. Kaggle. <https://www.kaggle.com/datasets>